

Firm-Level Investment Responses to Oil Price Shocks

An End-to-End Empirical Research Workflow in Python and R

Shuhan Zhang

Contents

Project overview	2
Analytical workflow	2
1. Building the analytical dataset in Python	2
2. R project setup and validation	3
3. Constructing valid panel variables	3
4. Aligning monthly shocks with firm-specific fiscal years	4
5. Main fixed-effects models	5
6. Structural oil-shock decomposition	5
7. Cumulative dynamic effects	6
8. Selected robustness checks	6
8.1 Alternative investment outcome	6
8.2 Continuous and nonlinear energy intensity	7
9. Reproducible outputs	7
Files in this portfolio	7

Project overview

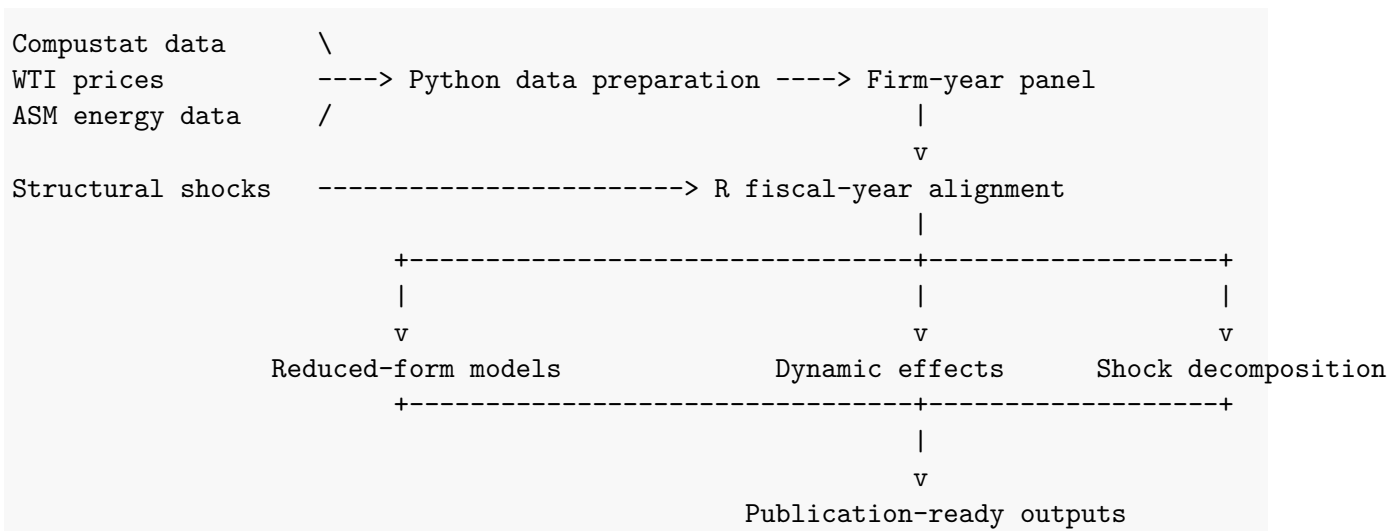
This project examines whether oil-price shocks reduce firm-level capital investment among U.S. manufacturing firms and whether the response is larger in energy-intensive industries. It demonstrates an end-to-end empirical workflow: Python is used to build the analytical firm-year panel, while R is used for firm-specific fiscal-year alignment, panel construction, econometric estimation, inference, and robustness analysis.

The workflow combines four data sources:

- Compustat firm-level financial statements;
- monthly West Texas Intermediate oil prices from FRED;
- four-digit NAICS energy-intensity measures constructed from the Annual Survey of Manufactures; and
- monthly structural oil shocks from Baumeister and Hamilton.

The proprietary Compustat observations are not included. The code nevertheless uses relative paths, explicit input checks, and documented variable definitions so that the analysis can be reproduced when authorized data are available.

Analytical workflow



1. Building the analytical dataset in Python

The companion script, `01_data_preparation.py`, performs the raw-data stage. It filters the Compustat extract to manufacturing firms, constructs investment outcomes and financial controls, handles gaps in the unbalanced panel, builds annual WTI changes, constructs industry energy intensity, validates merge keys, and exports the two processed files used in R.

A central data-quality rule is that a shifted value is treated as a one-year lag only when the prior observation is from the immediately preceding fiscal year:

```
def consecutive_lag(data, value_column, group_column="gvkey", year_column="fyear"):
    """Create a one-year lag without carrying values across panel gaps."""
    lagged_value = data.groupby(group_column)[value_column].shift(1)
    lagged_year = data.groupby(group_column)[year_column].shift(1)
    return lagged_value.where(data[year_column].eq(lagged_year + 1))
```

This prevents observations separated by multiple years from being incorrectly interpreted as consecutive records.

2. R project setup and validation

The full statistical workflow is contained in `02_panel_analysis.R`. The script first checks package availability, defines portable project paths, and validates that required variables and firm-year keys are present before estimation.

```
assert_unique_panel <- function(data, firm_id, year_id) {
  duplicated_key <- duplicated(data[c(firm_id, year_id)])

  if (any(duplicated_key)) {
    stop("The analytical panel contains duplicate firm-year observations.")
  }

  invisible(data)
}
```

These checks make assumptions about the analytical unit explicit and cause the script to fail early when an input file is inconsistent with the research design.

3. Constructing valid panel variables

Energy-intensive industries are defined using the 75th percentile of the industry energy-intensity distribution. The preferred reduced-form exposure is the interaction between a positive oil-price shock and this predetermined industry indicator. Firm controls enter with a one-year lag to reduce mechanical simultaneity with current investment.

```
energy_cutoff <- quantile(
  firm_panel$energy_intensity,
  probs = 0.75,
  na.rm = TRUE
)
```

```

firm_panel <- firm_panel |>
  mutate(
    high_energy = as.integer(energy_intensity >= energy_cutoff),
    oil_shock_positive = pmax(oil_shock, 0),
    oil_positive_high = oil_shock_positive * high_energy
  ) |>
  arrange(gvkey, fyear) |>
  group_by(gvkey) |>
  mutate(
    cash_at_w_l1 = consecutive_lag(cash_at_w, fyear),
    tangibility_w_l1 = consecutive_lag(tangibility_w, fyear)
  ) |>
  ungroup()

```

4. Aligning monthly shocks with firm-specific fiscal years

This is the most important data-engineering step in the R analysis. Firms have heterogeneous fiscal year-end dates, so a calendar-year average can assign monthly shocks to the wrong accounting period. The code constructs a 12-month window for every firm-year and uses a `data.table` non-equijoin to match monthly structural shocks to that window.

```

firm_fiscal_windows <- firm_panel |>
  mutate(
    fiscal_end_month = floor_date(datadate, unit = "month"),
    fiscal_start_month = fiscal_end_month %m-% months(11)
  )

firm_dt <- as.data.table(firm_fiscal_windows)
shock_dt <- as.data.table(monthly_shocks)

matched_months <- shock_dt[
  firm_dt,
  on = .(
    date >= fiscal_start_month,
    date <= fiscal_end_month
  ),
  allow.cartesian = TRUE,
  nomatch = 0L
]

```

The matched monthly observations are then aggregated within each firm-year. The script verifies the number of months matched before using the resulting exposures in the regression analysis.

5. Main fixed-effects models

The preferred specifications include firm and fiscal-year fixed effects with standard errors clustered by both firm and year. Firm fixed effects absorb persistent differences across companies, while year fixed effects absorb common macroeconomic shocks. Two-way clustering allows residual dependence within firms over time and across firms exposed to the same annual environment.

```
model_dynamic <- feols(  
  inv_at_w ~ oil_positive_high + oil_positive_high_l1 +  
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +  
    cash_at_w_l1 + tangibility_w_l1 |  
  gvkey + fyear,  
  data = dynamic_sample,  
  vcov = ~ gvkey + fyear  
)
```

The current-and-lagged model is estimated on a common complete-case sample so that coefficient changes across specifications are not driven by changes in the underlying observations.

6. Structural oil-shock decomposition

A change in the oil price can reflect different economic forces. The analysis therefore separates supply-driven shocks from shocks associated with global economic activity. Monthly structural shocks are aligned to firm fiscal years using the same procedure described above, interacted with high energy intensity, and entered in separate dynamic panel models.

```
model_supply_dynamic <- feols(  
  inv_at_w ~ supply_high_fy + supply_high_fy_l1 +  
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +  
    cash_at_w_l1 + tangibility_w_l1 |  
  gvkey + fyear,  
  data = decomposition_sample,  
  vcov = ~ gvkey + fyear  
)  
  
model_activity_dynamic <- feols(  
  inv_at_w ~ activity_high_fy + activity_high_fy_l1 +  
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +  
    cash_at_w_l1 + tangibility_w_l1 |  
  gvkey + fyear,  
  data = decomposition_sample,  
  vcov = ~ gvkey + fyear  
)
```

This step demonstrates how the empirical design moves beyond a single reduced- form price measure and investigates the underlying source of observed oil-price movements.

7. Cumulative dynamic effects

For models containing both current and lagged exposure, the economically relevant effect may be the sum of the two coefficients. A reusable helper computes the cumulative estimate and its standard error using the full variance-covariance matrix, including the covariance between coefficients.

```
cumulative_effect <- function(model, current_term, lagged_term) {  
  coefficients <- coef(model)  
  variance <- vcov(model)  
  
  estimate <- coefficients[current_term] + coefficients[lagged_term]  
  standard_error <- sqrt(  
    variance[current_term, current_term] +  
    variance[lagged_term, lagged_term] +  
    2 * variance[current_term, lagged_term]  
  )  
  
  tibble(  
    estimate = estimate,  
    standard_error = standard_error,  
    statistic = estimate / standard_error  
  )  
}
```

8. Selected robustness checks

The portfolio version intentionally limits robustness analysis to two checks that test distinct modeling choices.

8.1 Alternative investment outcome

The main outcome scales capital expenditures by beginning total assets. The first robustness model instead scales capital expenditures by beginning net property, plant, and equipment. This tests whether the result depends on the normalization used to measure investment.

```
model_alternative_outcome <- feols(  
  inv_ppent_w ~ oil_positive_high + oil_positive_high_l1 +  
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +  
    cash_at_w_l1 + tangibility_w_l1 |  
  gvkey + fyear,  
  data = alternative_outcome_sample,  
  vcov = ~ gvkey + fyear  
)
```

8.2 Continuous and nonlinear energy intensity

The second check replaces the high-energy indicator with continuous exposure and a natural-spline specification. It tests whether the finding is an artifact of imposing a 75th-percentile threshold and allows the response to vary nonlinearly across the energy-intensity distribution.

```
model_energy_spline <- feols(  
  inv_at_w ~ oil_spline_1 + oil_spline_2 + oil_spline_3 +  
    ln_at_l1 + lev_w_l1 + prof_w_l1 + sales_growth_w_l1 +  
    cash_at_w_l1 + tangibility_w_l1 |  
    gvkey + fyear,  
  data = functional_form_sample,  
  vcov = ~ gvkey + fyear  
)  
  
wald(model_energy_spline, keep = "oil_spline")
```

Together, these checks address outcome measurement and functional-form sensitivity without repeating a large number of closely related regressions.

9. Reproducible outputs

The R script exports labeled LaTeX tables, cumulative-effect summaries, and session information. Keeping output generation in the analytical script reduces manual transcription and ensures that tables remain synchronized with the estimated models.

```
etable(  
  list(  
    "Reduced form" = model_dynamic,  
    "Supply shock" = model_supply_dynamic,  
    "Activity shock" = model_activity_dynamic  
  ),  
  tex = TRUE,  
  file = here("output", "tables", "main_results.tex"),  
  replace = TRUE  
)  
  
writeLines(capture.output(sessionInfo()), here("output", "sessionInfo.txt"))
```

Files in this portfolio

- README.md: project overview, structure, and reproducibility notes;
- 01_data_preparation.py: raw-data preparation and analytical-panel creation;
- 02_panel_analysis.R: complete documented R analysis;
- 02_panel_analysis.Rmd: narrative portfolio notebook.